PYTHON PROGRAMMING — MID-TERM PROJECT REPORT

Track: Desktop Application (GUI)

PROJECT TITLE

Expense Tracker Desktop Application

Language: Python 3

GUI Library: tkinter

Storage: JSON (data.json)

Environment: Python virtual environment (.venv)

1. PROJECT OVERVIEW

This project is a personal Expense Tracker desktop application developed using Python and the tkinter GUI library. The main goal of this application is to help users easily record, manage, and analyze their daily expenses through a simple and user-friendly interface.

The project is designed to demonstrate core Python concepts, Object-Oriented Programming (OOP), data persistence using JSON, and a clean, organized project structure. It follows all the requirements given in the midterm project guidelines.

Main Objectives:

- Allow users to add, edit, delete, and view expense records
- Provide search, filter, and sorting options
- Show a live summary report (total, average, etc.)
- Store all data in a JSON file so it is not lost after closing the app
- Handle invalid inputs properly without crashing

2. PROJECT STRUCTURE

expense_tracker_project/

1. main.py → Entry point, starts the application
2. expense.py → Expense entity class
3. expense_manager.py → Business logic
4. storage.py → JSON file handling
5. gui_app.py → GUI controller
6. data.json → Stored expense data
7. requirements.txt → Dependencies
8. README.md → Setup instructions

3. OOP DESIGN — 4 CLASSES

1. Expense (Entity Class) — expense.py

This class represents a single expense record.

- Attributes: id, title, category, amount, date
- to_dict() → converts object into dictionary format
- from_dict() → creates an object from JSON data
- validate() → checks if data is correct before saving

2. JsonStorage (Storage Class) — storage.py

This class handles all file operations.

- save() → saves data into data.json
- load() → loads data from data.json
- get_next_id() → generates unique IDs

3. ExpenseManager (Service Class) — expense_manager.py

This is the main logic part of the application.

- add_expense() → adds new expense
- update_expense() → updates existing data
- delete_expense() → deletes a record
- search() → searches by title or category
- filter_by_category() → filters data
- sort_by() → sorts records
- get_report() → calculates total, average, highest, etc.

4. ExpenseApp (UI Controller) — gui_app.py

This class manages the graphical interface.

- _build_ui() → creates the layout
- _on_add(), _on_update(), _on_delete() → button actions
- _on_search(), _on_filter(), _on_sort() → filtering actions
- _refresh_table() → updates the display
- _update_report() → updates summary report

OOP Concepts Used

- Classes and constructors (__init__)
- Composition (App → Manager → Storage)
- Class methods and static methods
- Encapsulation (each class handles its own responsibility)

4. FEATURES

CRUD Operations

- Add new expense
- View all expenses
- Edit existing expense
- Delete selected expense

Search

- Search by title or category
- Case-insensitive search
- Shows a message if no result is found

Filter

- Filter by category using the dropdown
- Option to reset and view all records

Sort

- Sort by date, amount, or title
- Supports ascending and descending order

Summary Report

- Total expense
- Average expense
- Highest expense
- Total number of records
- Updates automatically after every change

Input Validation

- An empty title is not allowed
- Amount must be a number
- Amount must be greater than zero
- Date must follow YYYY-MM-DD format
- A warning is shown if no item is selected for update/delete

5. APPLICATION WORKFLOW

Startup

The program starts from main.py, loads data from JSON, and displays it in the table.

Add Expense

User fills the form → clicks Add → data is validated → saved → table updates

Edit Expense

User selects a record → data appears in form → updates it → saves → table refreshes

Delete Expense

User selects a record → clicks Delete → confirms → record removed → table updates

Search

User enters keyword → clicks Search → matching results are displayed

6. DATA PERSISTENCE

All data is stored in data.json in this format:


```
[
 {
   "id": 1,
   "title": "Lunch",
   "category": "Food",
   "amount": 350.0,
   "date": "2026-03-27"
 }
]
```


- Data is automatically saved after every operation
- Data is loaded when the app starts
- IDs remain consistent and are not reused

7. SETUP & RUN INSTRUCTIONS

Create virtual environment

python -m venv .venv

Activate (Windows)

.venv\Scripts\activate

Activate (macOS/Linux)

source .venv/bin/activate

Install dependencies

pip install -r requirements.txt

Run the application

python main.py

Generate requirements file

pip freeze > requirements.txt

8. MARKING CRITERIA COVERAGE

OOP Design (25%) → 4 classes, proper structure, clear responsibilities

Functionality (30%) → CRUD, search, filter, sort, reports

Persistence (15%) → JSON save/load with stable IDs

Environment (10%) → venv and requirements.txt

Validation (10%) → input checks and error handling

Clarity & Demo (10%) → clean UI and smooth workflow

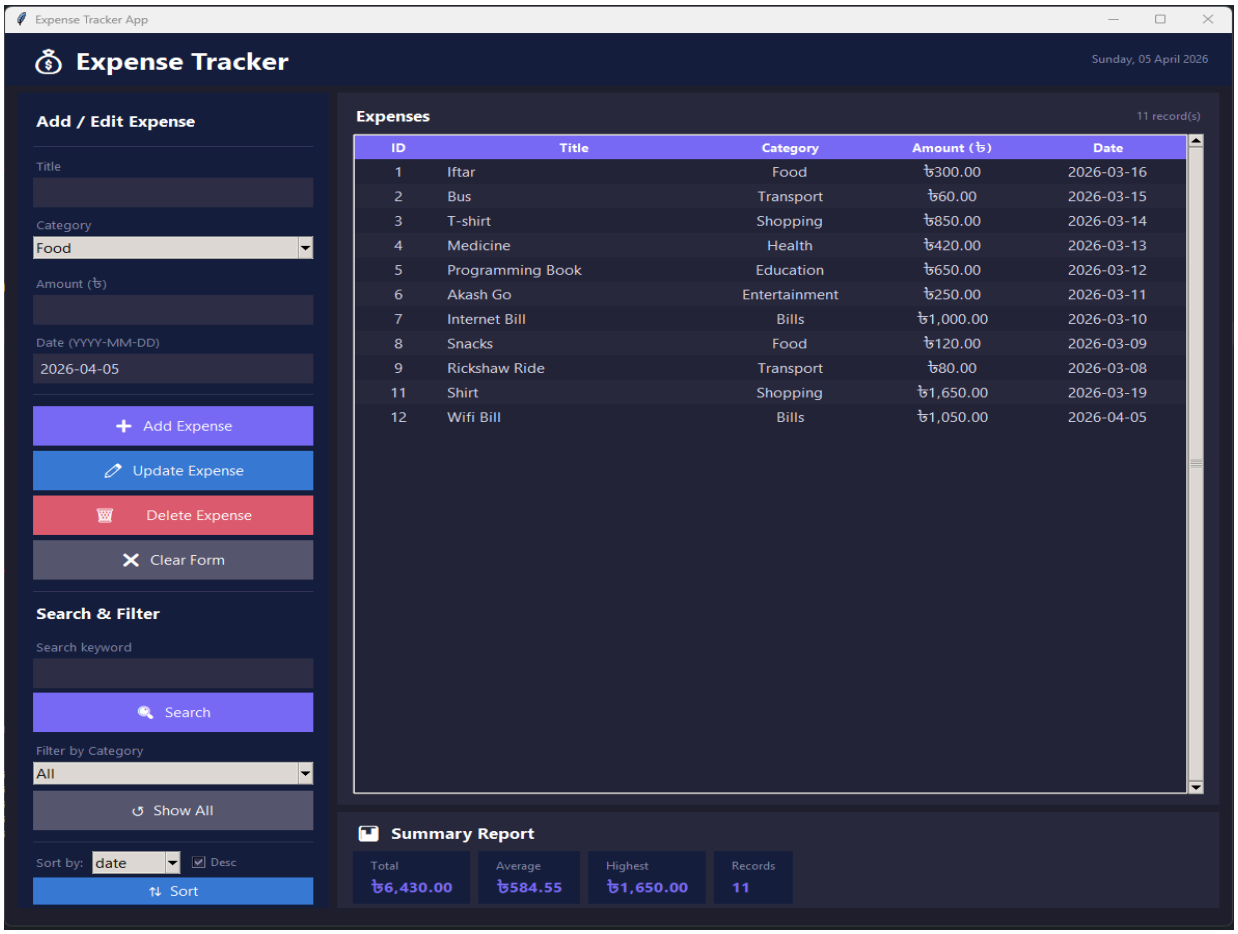
9. CONCLUSION

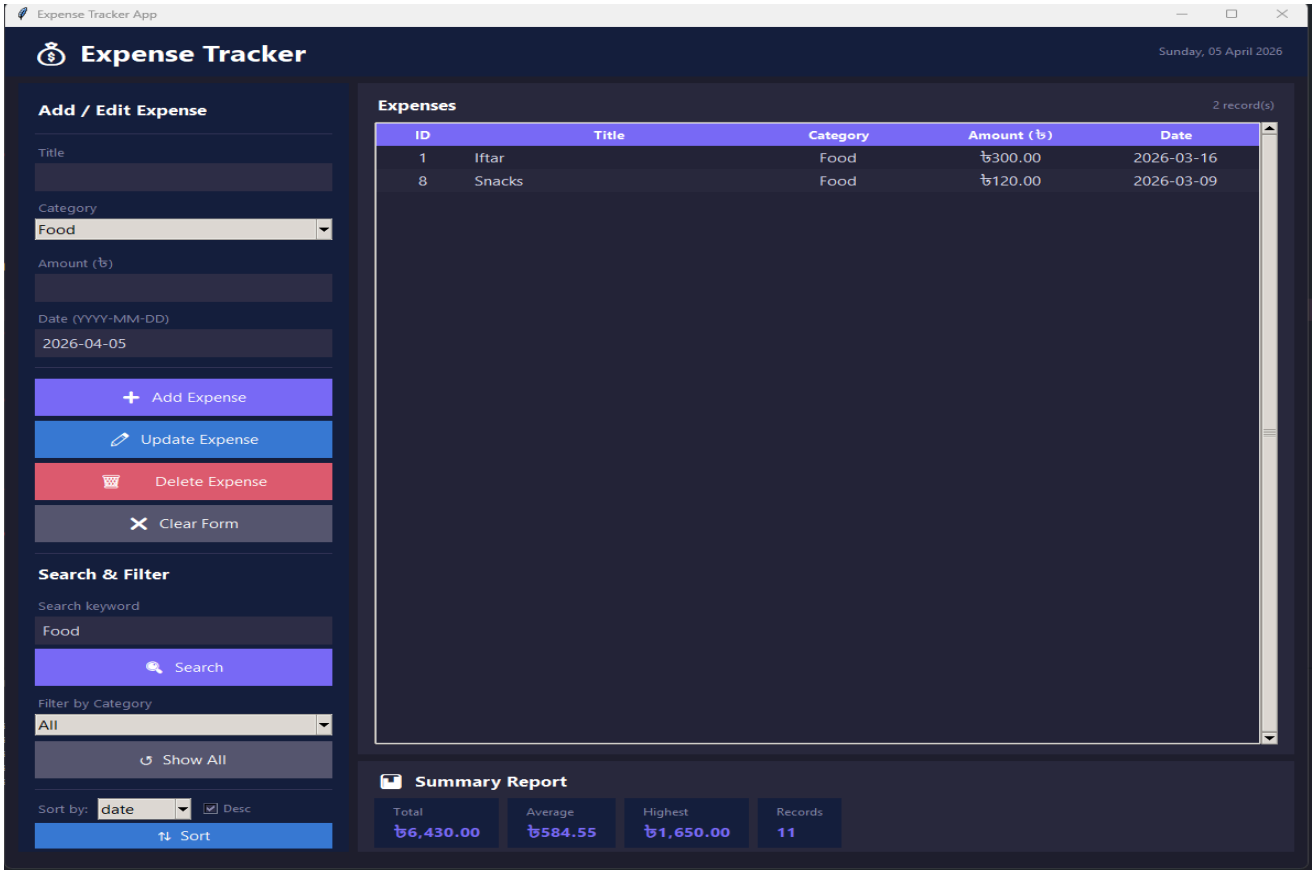
The Expense Tracker Desktop Application successfully fulfills all the requirements of the Python mid-term project. It demonstrates a strong understanding of OOP concepts, clean code structure, and real-world application development.

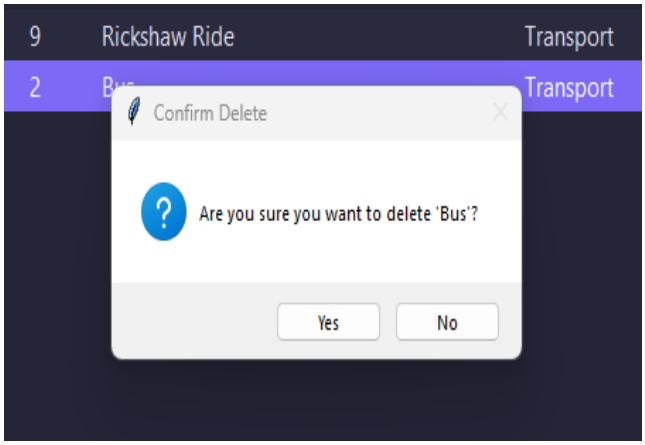
The use of multiple classes ensures separation of concerns, making the project easy to understand, maintain, and extend. The application is reliable, user-friendly, and practical for everyday use.

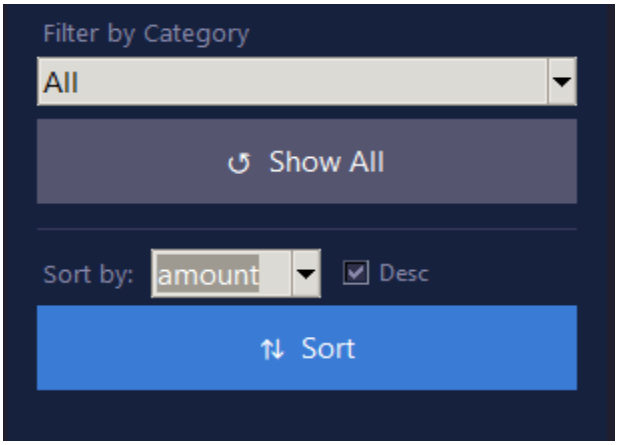
Overall, this project provides a solid foundation for building more advanced software applications in the future.

10. Project Screenshots/Demo




Expense Tracker App
Expense Tracker
Sunday, 05 April 2026
Add / Edit Expense
Title
Category
Food
Amount (৳)
Date (YYYY-MM-DD)
2026-04-05
Add Expense
Update Expense
Delete Expense
Clear Form
Search & Filter
Search keyword
Food
Search
Filter by Category
All
Show All
Sort by:
date
Desc
Sort
Expenses
2 record(s)
ID
Title
Category
Amount (৳)
Date
1
Iftar
Food
৳300.00
2026-03-16
8
Snacks
Food
৳120.00
2026-03-09
Summary Report
Total
৳6,430.00
Average
৳584.55
Highest
৳1,650.00
Records
11


9
Rickshaw Ride
Transport
2
Transport
Confirm Delete
Are you sure you want to delete 'Bus'?
Yes
No


Filter by Category
All
Show All
Sort by:
amount
Desc
Sort

Summary Report
Total
৳6,430.00
Average
৳584.55
Highest
৳1,650.00
Records
11